

What is a shape generator?

A shape generator is a function that takes your data array and returns an SVG path string. You pass it your data, it gives you back something like:

None

```
"M 60,278 L 160,265 L 260,232 L 360,195..."
```

That string goes directly into a `<path>` element's `d` attribute. D3 handles the math, Svelte handles the rendering — same pattern as scales.

d3.line()

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
    { month: "May", temp: 75 },
    { month: "Jun", temp: 83 },
    { month: "Jul", temp: 88 },
    { month: "Aug", temp: 86 },
    { month: "Sep", temp: 78 },
    { month: "Oct", temp: 67 },
    { month: "Nov", temp: 55 },
    { month: "Dec", temp: 44 }
  ]
```

```

const width = 600
const height = 400
const margin = { top: 20, right: 20, bottom: 40, left: 50 }

const xScale = d3.scalePoint()
  .domain(data.map(d => d.month))
  .range([margin.left, width - margin.right])

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])

const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
</svg>

```

`.x()` and `.y()` are accessor functions. They receive each data point and return the pixel position for that axis — the same logic you've been writing as inline attributes in `{#each}` loops. The generator calls them for you across the whole dataset and assembles the path string.

`scalePoint` is used here instead of `scaleBand` — it spaces categories evenly as points rather than bands, which is more appropriate for a line chart where you're connecting positions rather than drawing bars.

Adding dots

The path handles the line. Add a `{#each}` loop for dots at each data point.

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
    { month: "May", temp: 75 },
    { month: "Jun", temp: 83 },
    { month: "Jul", temp: 88 },
    { month: "Aug", temp: 86 },
    { month: "Sep", temp: 78 },
    { month: "Oct", temp: 67 },
    { month: "Nov", temp: 55 },
    { month: "Dec", temp: 44 }
  ]

  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  const xScale = d3.scalePoint()
    .domain(data.map(d => d.month))
    .range([margin.left, width - margin.right])

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data, d => d.temp)])
    .range([height - margin.bottom, margin.top])

  const lineGenerator = d3.line()
    .x(d => xScale(d.month))
    .y(d => yScale(d.temp))
```

```

</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />

  {#each data as d}
    <circle
      cx={xScale(d.month)}
      cy={yScale(d.temp)}
      r="4"
      fill="white"
      stroke="steelblue"
      stroke-width="2"
    />
  {/each}
</svg>

```

d3.area()

Works the same way as `d3.line()` but produces a filled shape between a baseline and your data line.

```

None

<script>
  import * as d3 from 'd3'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },

```

```
{ month: "Apr", temp: 65 },
{ month: "May", temp: 75 },
{ month: "Jun", temp: 83 },
{ month: "Jul", temp: 88 },
{ month: "Aug", temp: 86 },
{ month: "Sep", temp: 78 },
{ month: "Oct", temp: 67 },
{ month: "Nov", temp: 55 },
{ month: "Dec", temp: 44 }
]
```

```
const width = 600
const height = 400
```

```
const margin = { top: 20, right: 20, bottom: 40, left: 50 }
```

```
const xScale = d3.scalePoint()
  .domain(data.map(d => d.month))
  .range([margin.left, width - margin.right])
```

```
const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])
```

```
const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
```

```
const areaGenerator = d3.area()
  .x(d => xScale(d.month))
  .y0(height - margin.bottom) // baseline: bottom of chart area
  .y1(d => yScale(d.temp))    // data line: top of area
```

```
</script>
```

```
<svg viewBox="0 0 {width} {height}" {width} {height}>
  <!-- area first, so the line renders on top -->
  <path
```

```

    d={areaGenerator(data)}
    fill="steelblue"
    opacity="0.2"
  />
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
</svg>

```

`.y0()` is the baseline — usually the bottom of the chart area. `.y1()` is the data line. Everything between them gets filled.

Declarative vs. imperative: the same line chart

The scales and generators are identical in both versions. The only difference is how the output gets rendered.

Declarative

None

```

<script>
  import * as d3 from 'd3'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
    { month: "May", temp: 75 },
    { month: "Jun", temp: 83 }
  ]

```

```

const width = 600
const height = 400
const margin = { top: 20, right: 20, bottom: 40, left: 50 }

const xScale = d3.scalePoint()
  .domain(data.map(d => d.month))
  .range([margin.left, width - margin.right])

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])

const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
</svg>

```

Imperative

None

```

<script>
  import * as d3 from 'd3'
  import { onMount } from 'svelte'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
  ]

```

```

    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
    { month: "May", temp: 75 },
    { month: "Jun", temp: 83 }
  ]

  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  const xScale = d3.scalePoint()
    .domain(data.map(d => d.month))
    .range([margin.left, width - margin.right])

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data, d => d.temp)])
    .range([height - margin.bottom, margin.top])

  const lineGenerator = d3.line()
    .x(d => xScale(d.month))
    .y(d => yScale(d.temp))

  onMount(() => {
    const svg = d3.select("#chart")
      .append("svg")
      .attr("viewBox", `0 0 ${width} ${height}`)
      .attr("width", width)
      .attr("height", height)

    svg.append("path")
      .datum(data)
      .attr("d", lineGenerator)
      .attr("fill", "none")
      .attr("stroke", "steelblue")
      .attr("stroke-width", 2)
  })

```



```
</script>
```

```
<div id="chart"></div>
```

The `onMount` is necessary in the imperative version because D3 needs the DOM to exist before it can select and write into it. In the declarative version Svelte handles that automatically.

Axes

Axes follow the same pattern. D3 gives you the data — tick values and their positions — and you decide how to render it.

Declarative

Use `scale.ticks()` to get an array of tick values, then loop over them with `{#each}`.

None

```
<script>
  import * as d3 from 'd3'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
    { month: "May", temp: 75 },
    { month: "Jun", temp: 83 },
    { month: "Jul", temp: 88 },
    { month: "Aug", temp: 86 },
    { month: "Sep", temp: 78 },
    { month: "Oct", temp: 67 },
    { month: "Nov", temp: 55 },
    { month: "Dec", temp: 44 }
  ]
```

```

const width = 600
const height = 400
const margin = { top: 20, right: 20, bottom: 40, left: 50 }

const xScale = d3.scalePoint()
  .domain(data.map(d => d.month))
  .range([margin.left, width - margin.right])

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])

const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  <!-- line -->
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />

  <!-- x axis labels -->
  {#each data as d}
    <text
      x={xScale(d.month)}
      y={height - margin.bottom + 20}
      text-anchor="middle"
      font-size="12"
    >{d.month}</text>
  {/each}

```

```

<!-- y axis gridlines and labels -->
{#each yScale.ticks(5) as tick}
  <line
    x1={margin.left}
    x2={width - margin.right}
    y1={yScale(tick)}
    y2={yScale(tick)}
    stroke="#e5e5e5"
  />
  <text
    x={margin.left - 8}
    y={yScale(tick)}
    text-anchor="end"
    dominant-baseline="middle"
    font-size="12"
  >{tick}</text>
{/each}
</svg>

```

`yScale.ticks(5)` asks the scale for approximately 5 evenly-spaced tick values within the domain. The scale picks round numbers automatically.

`text-anchor="middle"` centers text horizontally on its x position.

`dominant-baseline="middle"` centers it vertically on its y position.

Imperative

None

```

<script>
  import * as d3 from 'd3'
  import { onMount } from 'svelte'

  const data = [
    { month: "Jan", temp: 40 },
    { month: "Feb", temp: 44 },
    { month: "Mar", temp: 54 },
    { month: "Apr", temp: 65 },
  ]

```

```
{ month: "May", temp: 75 },
{ month: "Jun", temp: 83 },
{ month: "Jul", temp: 88 },
{ month: "Aug", temp: 86 },
{ month: "Sep", temp: 78 },
{ month: "Oct", temp: 67 },
{ month: "Nov", temp: 55 },
{ month: "Dec", temp: 44 }
]

const width = 600
const height = 400
const margin = { top: 20, right: 20, bottom: 40, left: 50 }

const xScale = d3.scalePoint()
  .domain(data.map(d => d.month))
  .range([margin.left, width - margin.right])

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])

const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))

onMount(() => {
  const svg = d3.select("#chart")
    .append("svg")
    .attr("viewBox", `0 0 ${width} ${height}`)
    .attr("width", width)
    .attr("height", height)

  svg.append("path")
    .datum(data)
    .attr("d", lineGenerator)
```

```
.attr("fill", "none")
.attr("stroke", "steelblue")
.attr("stroke-width", 2)

svg.append("g")
  .attr("transform", `translate(0, ${height - margin.bottom})`)
  .call(d3.axisBottom(xScale))

svg.append("g")
  .attr("transform", `translate(${margin.left}, 0)`)
  .call(d3.axisLeft(yScale).ticks(5))
})
</script>

<div id="chart"></div>
```

D3 writes the tick marks, tick lines, and labels directly into the `<g>` elements. You get sensible defaults quickly, but customizing them means overriding D3's generated styles and elements rather than editing your own markup.

Which to use

Both produce the same output. The declarative approach is more consistent with how you've been working and easier to customize. The imperative approach is faster to drop in and is what you'll see in most examples in the wild. As you get comfortable reading both, you'll develop a feel for when each is the right tool.